

load_historical.py

```
"""
load_historical.py

One-time / monthly importer to push historical NVD JSON files
(2010?present)
from ./data/nvd/historical/ into your Render Postgres DB.

Usage:
$ python load_historical.py
"""

import os
import json
import gzip
from typing import Dict, Any, List, Optional
from database.db_manager import db_manager
import config

HIST_DIR = config.NVD_HISTORICAL_DIR

def find_year_files() -> Dict[int, str]:
    patterns = [
        "CVE-{year}.json",
        "nvdcve-1.1-{year}.json",
        "nvdcve-2.0-{year}.json",
        "CVE-{year}.json.gz",
        "nvdcve-1.1-{year}.json.gz",
        "nvdcve-2.0-{year}.json.gz",
    ]
    files: Dict[int, str] = {}
    if not os.path.exists(HIST_DIR):
        print(f"[Import] No directory: {HIST_DIR}")
        return files
    years = list(range(2010, 2051))
    for y in years:
        for p in patterns:
            fp = os.path.join(HIST_DIR, p.format(year=y))
            if os.path.exists(fp):
                files[y] = fp
                break
    return files

def parse_vulns_from_file(path: str) -> List[Dict[str, Any]]:
    open_f = gzip.open if path.endswith(".gz") else open
    try:
        with open_f(path, "rt", encoding="utf-8", errors="ignore") as f:
            data = json.load(f)
    except Exception as e:
        print(f"[Import] Failed to read {path}: {e}")
        return []
    vulns = data.get("vulnerabilities") or data.get("CVE_Items") or []
    out: List[Dict[str, Any]] = []
    for item in vulns:
        cve_obj = item.get("cve", item)
        processed = normalize_cve(cve_obj)
        if processed:
            out.append(processed)
    return out

def normalize_cve(cve_data: Dict[str, Any]) -> Optional[Dict[str, Any]]:
```

```

try:
cve_id = cve_data.get("id") or cve_data.get("CVE_data_meta", {}).get("ID")
if not cve_id:
return None

# Description
description = ""
desc = cve_data.get("descriptions") or cve_data.get("description",
{}).get("description_data")
if isinstance(desc, list) and desc:
description = desc[0].get("value") or ""
elif isinstance(desc, str):
description = desc

# Severity / CVSS
severity = "UNKNOWN"
cvss_score = None
metrics = cve_data.get("metrics") or {}
for key in ["cvssMetricV31", "cvssMetricV30", "cvssMetricV2",
"cvssMetricV3"]:
arr = metrics.get(key)
if isinstance(arr, list) and arr:
m = arr[0].get("cvssData") or arr[0].get("baseMetricV3", {}).get("cvssV3",
{})
if isinstance(m, dict):
cvss_score = m.get("baseScore")
sev = m.get("baseSeverity") or arr[0].get("baseSeverity")
if isinstance(sev, str):
severity = sev.upper()
break

# CWE
cwe = "Unknown"
weaknesses = cve_data.get("weaknesses") or []
if weaknesses:
d = weaknesses[0].get("description") or weaknesses[0].get("description",
[])
if isinstance(d, list) and d:
cwe = d[0].get("value", "Unknown")

# Dates
published = cve_data.get("published") or cve_data.get("publishedDate") or
""
last_modified = cve_data.get("lastModified") or
cve_data.get("lastModifiedDate") or ""

# References
refs = []
references = cve_data.get("references") or cve_data.get("reference_data",
[])
if isinstance(references, list):
for r in references:
url = r.get("url") or r.get("refsource")
if url:
refs.append(url)

return {
"ID": cve_id,
"Description": description or "No description available",
"Severity": severity,
"CVSS_Score": cvss_score,
"CWE": cwe,
"Published": published or "",
"lastModified": last_modified or "",

```

```

"References": refs,
"Products": [],
"metrics": {},
}
except Exception as e:
print(f"[Import] Error processing CVE item: {e}")
return None

def main():
if not db_manager.use_database:
print("[Import] ERROR: DATABASE_URL missing. Set it in your environment.")
return

year_files = find_year_files()
if not year_files:
print(f"[Import] No historical files in {HIST_DIR}")
return

total_inserted = 0
for year in sorted(year_files.keys()):
path = year_files[year]
print(f"[Import] Loading {year} from {os.path.basename(path)}")

# If DB already has this year, skip heavy parse (fast path)
existing = db_manager.get_cves_by_filter(year=str(year), limit=1)
if existing:
print(f"[Import] DB already has data for {year}; skipping file.")
continue

rows = parse_vulns_from_file(path)
if not rows:
print(f"[Import] No parsed CVEs for {year}")
continue

# Save in batches
print(f"[Import] Inserting {len(rows)} CVEs for {year}...")
ok = db_manager.save_cves_batch(rows)
if ok:
total_inserted += len(rows)
print(f"[Import] ? Saved {len(rows)} CVEs for {year}")
else:
print(f"[Import] ?? Failed to save some CVEs for {year}")

print(f"[Import] Done. Inserted ~{total_inserted} CVEs total.")

if __name__ == "__main__":
main()

```